

AWS DOP-C02 — Complete Exam Study Guide

Keyword → Answer Map | Scenario Patterns | Traps | Multi-Region HA

Exam weight by domain (official AWS breakdown):

- Domain 1: SDLC Automation (CI/CD) — **22%**
- Domain 2: Configuration Management & IaC — **17%**
- Domain 3: Resilient Cloud Solutions — **15%**
- Domain 4: Monitoring & Logging — **15%**
- Domain 5: Incident & Event Response — **14%**
- Domain 6: Security & Compliance — **17%**

Total: 75 questions | 180 minutes | Passing score: 750/1000

DOMAIN 1 — SDLC Automation / CI/CD (22%)

Highest weighted domain. Know every service cold.

1.1 The Core CI/CD Trinity

CodeCommit — AWS managed Git repository

- Stores source code
- Triggers pipelines via EventBridge on push/PR
- Integrates with CodePipeline natively
- Note: AWS deprecated new CodeCommit sign-ups in 2024. Exam still tests it.
GitHub/Bitbucket = CodeStar Connections.

CodeBuild — Managed build service

- Runs buildspec.yml — defines build phases: install, pre_build, build, post_build
- Outputs artifacts to S3
- Can run Docker builds, unit tests, static analysis
- Environment: managed image OR custom Docker image

- IAM role on CodeBuild project needs: ecr:GetAuthorizationToken, s3:PutObject, logs:CreateLogGroup

CodeDeploy — Deployment automation

- Deploys to: EC2, On-premises, ECS (blue/green), Lambda (canary/linear)
- Uses appspec.yml to define deployment hooks
- Deployment hooks (EC2): BeforeInstall, AfterInstall, ApplicationStart, ValidateService
- Automatic rollback triggered by: CloudWatch Alarm OR deployment failure

CodePipeline — Orchestration

- Stages: Source → Build → Test → Deploy (and optional Manual Approval)
- Artifact store: S3 (always)
- Cross-region actions: can deploy to multiple regions from one pipeline
- Cross-account: use separate CodeDeploy role with cross-account trust

1.2 CI/CD Keyword → Answer Table

When you see this keyword	Answer
"automated pipeline, native AWS"	CodePipeline + CodeBuild + CodeDeploy
"rollback automatically on alarm"	CodeDeploy + CloudWatch Alarm auto-rollback
"zero downtime EC2 deployment"	CodeDeploy blue/green
"canary deployment for Lambda"	CodeDeploy Lambda deployment group (Canary10Percent5Minutes)
"gradual traffic shift Lambda"	CodeDeploy Linear strategy
"manual approval before prod"	CodePipeline Manual Approval stage → SNS notification
"GitHub/Bitbucket as source"	CodeStar Connections
"build Docker image, push to registry"	CodeBuild + ECR
"store pipeline artifacts"	S3 (always, never EFS or EBS)

When you see this keyword	Answer
"trigger pipeline on code push"	EventBridge rule on CodeCommit push event
"feature flags"	AppConfig (Systems Manager)
"safe config deployment with rollback"	AppConfig with deployment strategy
"infrastructure as code"	CloudFormation
"deploy stack to 100 accounts"	CloudFormation StackSets
"preview changes before applying"	CloudFormation Change Sets
"reuse CloudFormation components"	Nested stacks OR CloudFormation modules
"Jenkins already in use"	CodePipeline (orchestrates) + Jenkins (build step)
"Terraform already in use"	Mention it but CloudFormation is the AWS-native answer

1.3 CodeDeploy Deployment Types — Know the Difference

EC2 / On-premises:

In-place (rolling)

- Same instances updated in batches
- Brief downtime possible during batch update
- Faster (no new instances to provision)
- Rollback = re-deploy previous version (slow)
- Use when: cost matters, some downtime acceptable

Blue/Green

- New ASG (green) provisioned, ALB shifts traffic from old (blue) to new
- Zero downtime guaranteed
- Instant rollback = point ALB back to blue
- Old instances kept for configurable time then terminated
- Use when: "zero downtime" or "instant rollback" in question

ECS Blue/Green:

- Two ECS task sets behind one ALB
- Traffic shifts via weighted target groups
- Integrates with CodeDeploy ECS deployment type

Lambda:

- **Canary** — e.g., `Canary10Percent5Minutes`: 10% to new version for 5 min, then 100%
 - **Linear** — e.g., `Linear10PercentEvery1Minute`: increase by 10% every minute
 - **All-at-once** — instant switch (no canary, no validation period)
 - Rollback = CloudWatch alarm on error rate fires → CodeDeploy rolls back alias
-

1.4 appspec.yml vs buildspec.yml

buildspec.yml — used by CodeBuild

```
version: 0.2
phases:
  install:
    commands:
      - npm install
  build:
    commands:
      - npm test
      - npm run build
artifacts:
  files:
    - '**/*'
```

appspec.yml — used by CodeDeploy

- For EC2: defines files to copy + lifecycle hook scripts
- For Lambda: defines function name, alias, and hook functions
- For ECS: defines task definition and container port mappings

Lifecycle hooks for EC2 (in order):

ApplicationStop → DownloadBundle → BeforeInstall → Install → AfterInstall → ApplicationStart
→ ValidateService

1.5 Elastic Beanstalk — When to Use

Elastic Beanstalk = PaaS. You upload code, AWS manages infrastructure.

Deployment policies:

- All at once — fastest, downtime during deploy
- Rolling — batch by batch, reduced capacity during deploy
- Rolling with additional batch — extra instances added first, no capacity reduction
- Immutable — new instances launched in new ASG, then swapped (safest, slowest)
- Blue/Green — swap environment URLs (DNS level, not instance level)
- Traffic splitting — canary testing (% of traffic to new version)

When exam says Beanstalk:

- "Developers just want to deploy code without managing servers" → Beanstalk
- "Fastest rollback" → Immutable or Blue/Green
- "No capacity reduction during rolling" → Rolling with additional batch

Trap: Beanstalk blue/green uses DNS swap (CNAME swap). CodeDeploy blue/green uses ALB target group swap. They are different mechanisms.

1.6 Container Deployments

ECS with CodeDeploy (blue/green):

- Two task sets: blue (current) and green (new)
- ALB listener shifts traffic
- appspec.yml defines TaskDefinition and LoadBalancerInfo
- Hooks: BeforeInstall, AfterInstall, AfterAllowTestTraffic, BeforeAllowTraffic, AfterAllowTraffic

EKS deployments:

- Helm charts for packaging
- kubectl apply for manifest deployment
- CodePipeline can invoke kubectl via CodeBuild step
- No native CodeDeploy support for EKS — always CodeBuild + kubectl

ECR (Elastic Container Registry):

- Private Docker registry
 - Image scanning: basic (on push) or enhanced (Inspector continuous)
 - Lifecycle policies: auto-delete old/untagged images
 - Cross-account: resource-based policy on ECR repo
 - Cross-region replication: ECR replication rules
-

DOMAIN 2 — Configuration Management & IaC (17%)

2.1 CloudFormation Deep Dive

Key concepts:

Stack — single deployment unit of CloudFormation resources

StackSets — deploy same template to multiple accounts × regions

- Requires: administrator account + execution role in target accounts
- With Organizations = auto-deploy to new accounts on join
- Deployment options: max concurrent accounts, failure tolerance %

Change Sets — preview what will change before applying

- Never skip for production changes
- Shows: Add / Modify / Remove for each resource
- Does NOT execute — you review then execute separately

Drift detection — find resources changed outside CloudFormation

- Run: detect-stack-drift → check drift status
- Drifted resources: CloudFormation state ≠ actual state

CloudFormation helper scripts (on EC2):

- cfn-init: reads AWS::CloudFormation::Init metadata, installs packages/files/services
- cfn-signal: signals CloudFormation when instance is ready (used with CreationPolicy)
- cfn-hup: polls for stack updates and applies config changes to running instances
- cfn-get-metadata: retrieves metadata

DependsOn — explicit dependency ordering between resources

WaitCondition / CreationPolicy — wait for EC2 to signal success before marking stack complete

Custom Resources — invoke Lambda on stack create/update/delete for resources

CloudFormation doesn't natively support

Stack policies — prevent accidental updates to specific resources (e.g., production RDS)

2.2 Systems Manager (SSM) — Full Breakdown

SSM is heavily tested in DOP. Know every sub-service.

SSM Agent — must be installed and running on EC2. Pre-installed on Amazon Linux 2, Ubuntu 16.04+, Windows Server 2008+. Needs IAM role with AmazonSSMManagedInstanceCore policy.

SSM Sub-services:

Session Manager

- Browser-based or CLI shell access to EC2
- No SSH, no bastion, no open inbound ports required
- Audit: session logs to S3 or CloudWatch Logs
- Replaces bastion hosts completely

Run Command

- Execute documents/scripts on one or thousands of instances
- No SSH needed

- Output: S3 or CloudWatch Logs
- Rate control: concurrency % and error threshold %
- Use for: one-time scripts, software install, configuration at scale

Patch Manager

- Automated OS patching
- Patch baseline: defines approved/rejected patches per OS
- Maintenance window: defines when patching runs
- Compliance reporting: patch compliance visible in SSM console
- Patch groups: tag instances with Patch Group tag to assign baseline

State Manager

- Ensures instances maintain desired state continuously
- Association: links document to instances on a schedule
- Example: ensure CloudWatch agent always installed, antivirus always running

Parameter Store

- Key-value store for config and secrets
- Tiers: Standard (free, 4KB) and Advanced (paid, 8KB, parameter policies)
- Types: String, StringList, SecureString (KMS encrypted)
- Hierarchy: /app/env/key (e.g., /myapp/prod/db_password)
- Parameter policies (Advanced): expiration, notification, no-change alerts
- Limitation: NO automatic rotation (use Secrets Manager for that)

Automation

- Run automation runbooks (documents) for operational tasks
- Triggered by: Config rules, EventBridge, manual, scheduled
- Common runbooks: AWS-StopEC2Instance, AWS-CreateSnapshot, AWS-PatchInstanceWithRollback
- Use for: auto-remediation of Config rule violations

Inventory

- Collects metadata from instances: installed software, network config, Windows updates

- Query with Resource Data Sync to S3 → Athena for analysis

AppConfig

- Deploy application configuration safely with validation and rollback
 - Deployment strategies: Linear, Exponential, AllAtOnce
 - Validators: JSON Schema validator or Lambda validator
 - Use when: feature flags, dynamic config, safe config rollout with rollback capability
 - Difference from Parameter Store: AppConfig adds deployment strategy + validation
-

2.3 AWS Config

What it does: Records configuration of AWS resources over time. Evaluates compliance against rules.

Config Rules:

- AWS Managed Rules: pre-built (e.g., required-tags, restricted-ssh, s3-bucket-public-read-prohibited)
- Custom Rules: Lambda function that evaluates resources
- Evaluation triggers: configuration change OR periodic

Auto-remediation:

- Config Rule → SSM Automation document
- Example: S3 bucket goes public → Config detects → triggers SSM Automation → makes bucket private

Conformance Packs:

- Collection of Config rules + remediation actions as a single deployable unit
- Deploy across org with StackSets

Aggregator:

- Centralises Config data from multiple accounts/regions into one view
- Requires authorisation from source accounts

Key use cases:

- "Detect and auto-remediate non-compliant resources" → Config + SSM Automation
 - "Prove compliance to auditors" → Config conformance pack
 - "Track who changed a security group" → Config history (shows what changed) + CloudTrail (shows who changed it)
-

2.4 OpsWorks

Managed Chef and Puppet. Niche but tested occasionally.

- OpsWorks Stacks: Chef recipes, layer-based architecture
- OpsWorks for Chef Automate: fully managed Chef server
- OpsWorks for Puppet Enterprise: fully managed Puppet server

When exam mentions: "existing Chef/Puppet infrastructure" OR "configuration management with recipes/manifests" → OpsWorks

DOMAIN 3 — Resilient Cloud Solutions (15%)

3.1 Multi-Region HA — The Master Reference

Global Services (no region selection — always available everywhere):

Route 53

- Global DNS — authoritative DNS for your domain
- Routing policies:
 - Simple: one record, one destination
 - Failover: primary + secondary, health check drives switch
 - Latency: routes to region with lowest latency for the user
 - Weighted: distribute traffic by percentage (10% to new, 90% to old)
 - Geolocation: route by user's country/continent
 - Geoproximity: route by geographic distance with bias

- Multivalue: multiple healthy endpoints returned
- Health checks: HTTP/HTTPS/TCP, checks endpoint every 10-30 seconds
- Calculated health checks: combine multiple checks with AND/OR logic
- Private hosted zone: DNS for VPC internal resolution
- **In every multi-region HA answer, Route 53 is layer 1**

CloudFront

- Global CDN with 400+ edge locations
- Origins: S3, ALB, EC2, API Gateway, custom HTTP
- Origin failover: primary fails → automatic switch to secondary origin
- Behaviours: path-based routing to different origins (/api/* → ALB, /* → S3)
- Cache: TTL-based, can invalidate with CreateInvalidation
- Functions: CloudFront Functions (lightweight JS at edge) or Lambda@Edge (full Node.js/Python)
- Signed URLs/cookies: restrict access to premium content
- OAC (Origin Access Control): restrict S3 bucket to CloudFront only (replaces OAI)
- **"Global static website HA" = S3 + CloudFront, always**

Global Accelerator

- Two static anycast IP addresses (never change)
- Routes TCP/UDP to nearest healthy AWS endpoint
- No caching — pure routing/acceleration
- Works for: EC2, ALB, NLB, EIP
- 60% faster than public internet routing (uses AWS backbone)
- **Use when: "static IPs required" OR non-HTTP (gaming, IoT, VoIP) OR "instant failover with same IP"**
- **Trap: not CloudFront. Global Accelerator does NOT cache.**

IAM

- Users, groups, roles, policies are ALL global
- One IAM role works in every region
- **Trap: people assume IAM is regional. It is not.**

AWS Organizations

- Manages multiple AWS accounts centrally
- Service Control Policies (SCP): guardrails on what accounts can do maximum
- SCPs do NOT grant permissions — they only restrict
- Tag policies: enforce consistent tagging across org
- Backup policies: enforce backup plans across org
- AI services opt-out policies: opt out of AI data usage org-wide

Multi-Region Services (requires configuration):

DynamoDB Global Tables

- Multi-master, multi-region replication
- Write from ANY region — replicates to all others in ~1 second
- Read from local region = low latency
- Conflict resolution: last-writer-wins
- Requires: DynamoDB Streams enabled
- **Use when: "NoSQL" + "write from multiple regions" + "global low latency"**
- **The single best answer for globally distributed, high-availability NoSQL**

Aurora Global Database

- One primary region (read/write) + up to 5 secondary regions (read-only)
- Replication lag: under 1 second (uses dedicated replication infrastructure)
- Failover: promote secondary to primary in under 1 minute
- RPO: ~1 second | RTO: ~1 minute
- **Use when: "relational" + "multi-region" + "fast failover" + "read scale globally"**

RDS Read Replicas (cross-region)

- Async replication from primary to replica in another region
- Can promote to standalone DB for DR
- Slower failover than Aurora Global
- **Use when: "RDS MySQL/PostgreSQL" (not Aurora) + "read scale in another region"**

- Cross-Region Replication (CRR): async replication of objects to bucket in another region
- Same-Region Replication (SRR): copy within region for compliance/aggregation
- Multi-Region Access Points: single global S3 endpoint, routes to nearest bucket
- Versioning must be enabled on both source and destination for CRR
- Replication Time Control (RTC): 99.99% of objects replicated within 15 minutes (with SLA)

Secrets Manager

- Cross-region secret replication: same secret available in multiple regions
- Each replica is independent — can be promoted to primary
- **Trap: Parameter Store does NOT replicate across regions. Secrets Manager does.**

CloudFormation StackSets

- Deploy same template to multiple accounts × multiple regions in one operation
- With Organizations: SERVICE_MANAGED mode — auto-deploys to new accounts
- Deployment options: failure tolerance, max concurrent deployments
- **Use when: "same infrastructure in all regions" or "all accounts automatically"**

CloudTrail Organization Trail

- One trail → captures API calls from ALL regions, ALL accounts in org
- Delivered to central S3 bucket
- **Use when: "centralised audit across all regions and accounts" → org trail**

CodePipeline Multi-Region

- Pipeline itself is regional
- Add cross-region deployment actions to deploy to multiple regions from one pipeline
- Artifact replication: CodePipeline creates replication buckets in each target region

Regional Services (common trap — people assume global):

Service	Scope	Trap
Systems Manager / Parameter Store	Regional	Parameters don't auto-replicate

Service	Scope	Trap
CodePipeline	Regional	Pipeline runs in one region
CodeBuild	Regional	Build jobs run in one region
ElastiCache	Regional	Cannot cache across regions natively
DynamoDB DAX	Regional	Accelerator cache is regional
SQS	Regional	Queues are regional
SNS	Regional	Topics are regional
KMS	Regional	CMKs are regional (multi-region keys available but explicit config)
ACM Certificates	Regional	Must request in each region (except CloudFront = us-east-1)

3.2 The Standard Multi-Region HA Stack

Memorise this stack. It appears in 3-4 questions in different forms:

```

Layer 1 – DNS:      Route 53 (failover routing + health checks)
Layer 2 – Edge:     CloudFront (caching + origin failover)
Layer 3 – Load:    ALB in each region
Layer 4 – Compute:  ASG + EC2 / ECS Fargate / Lambda (per region)
Layer 5 – Data:     Aurora Global DB (relational) OR DynamoDB Global Tables (NoSQL)
Layer 6 – Config:   Secrets Manager with cross-region replication
Layer 7 – IaC:      CloudFormation StackSets (deploy infra to all regions)

```

3.3 Disaster Recovery Patterns

Know these four in order of cost vs RTO/RPO:

Backup & Restore — cheapest, slowest

- RTO: hours | RPO: hours

- Regular snapshots/backups to S3, restore when disaster strikes
- No standby infrastructure

Pilot Light — minimal standby

- RTO: minutes to hours | RPO: minutes
- Core DB running in secondary region (Aurora Global / RDS replica)
- Other resources (EC2, ALB) exist but are stopped/minimal
- Scale up quickly when disaster strikes

Warm Standby — scaled-down but running

- RTO: minutes | RPO: seconds to minutes
- Full stack running in secondary region but at reduced capacity
- Scale up to production size on failover
- Route 53 health check triggers DNS failover

Multi-Site Active/Active — most expensive, fastest

- RTO: near zero | RPO: near zero
 - Full production capacity in multiple regions simultaneously
 - Route 53 routes traffic to all regions (weighted or latency)
 - DynamoDB Global Tables or Aurora Global for data
-

3.4 Auto Scaling

EC2 Auto Scaling:

- Scaling policies:
 - Target Tracking: maintain a metric at a target (e.g., CPU at 50%) — simplest, recommended
 - Step Scaling: scale by X instances when metric crosses threshold
 - Simple Scaling: scale and wait cooldown period (old, avoid)
 - Scheduled Scaling: scale at specific times (e.g., pre-scale for Monday 9am traffic)
 - Predictive Scaling: ML-based, forecasts future load (best for cyclical patterns)
- Launch Template vs Launch Configuration:

- Launch Template: newer, supports multiple instance types, Spot + On-demand mix, versioning
- Launch Configuration: old, cannot modify, no versioning, avoid for new setups
- Instance refresh: rolling update of instances to new launch template version
- Warm pools: pre-initialised instances that launch faster when scaling out
- Lifecycle hooks: pause instances during launch or termination for custom actions

Application Auto Scaling: scales ECS tasks, DynamoDB, Aurora replicas, Lambda concurrency

DOMAIN 4 — Monitoring & Logging (15%)

4.1 CloudWatch Deep Dive

Metrics:

- Default EC2 metrics: CPU, NetworkIn/Out, DiskReadOps (NOT memory, NOT disk space — need agent)
- Custom metrics: push with put-metric-data API
- High-resolution metrics: 1-second resolution (standard = 60 seconds)
- Metric Math: combine metrics with formulas (e.g., error rate = errors / requests × 100)
- Contributor Insights: identify top contributors to metrics (e.g., top 10 IPs causing errors)

Alarms:

- States: OK, ALARM, INSUFFICIENT_DATA
- Actions: SNS notification, EC2 action (stop/terminate/reboot/recover), Auto Scaling action
- Composite alarms: combine multiple alarms with AND/OR logic (reduces alarm noise)
- Anomaly detection: ML-based dynamic thresholds (no need to set fixed threshold)

Logs:

- Log group → Log stream → Log events
- Retention: 1 day to 10 years (default: never expire — always set retention!)
- CloudWatch Agent: collects logs AND custom metrics from EC2
- Metric filters: extract metrics from log patterns (e.g., count ERROR occurrences)

- CloudWatch Logs Insights: SQL-like query language for log analysis
- Subscription filters: stream logs to Lambda, Kinesis, Kinesis Firehose in real-time
- Export to S3: batch export (not real-time) for long-term archival → Athena analysis

Container insights: ECS/EKS performance metrics and logs with enhanced visibility

Lambda insights: detailed Lambda metrics including memory usage, cold starts

ServiceLens: distributed tracing combined with CloudWatch metrics and logs (uses X-Ray)

4.2 CloudTrail

- Records every API call in AWS: who, what, when, from where, on what resource
 - Event types:
 - Management events: control plane operations (create/delete/modify resources) — enabled by default
 - Data events: S3 object operations, Lambda invocations — disabled by default (high volume/cost)
 - Insights events: unusual API activity detection
 - Trail: delivers logs to S3 bucket (can also send to CloudWatch Logs)
 - Organization trail: single trail covering all accounts and regions in AWS org
 - Log file integrity: SHA-256 hashing + signing, validate with validate-logs CLI
 - Encryption: SSE-S3 by default, SSE-KMS for stronger control
 - **Trap: CloudTrail is NOT real-time. Logs delivered to S3 within ~15 minutes.**
 - For near-real-time: CloudTrail → CloudWatch Logs → metric filter + alarm → SNS
-

4.3 AWS X-Ray

- Distributed tracing for applications
- Trace: end-to-end request path across services
- Segment: work done by one service
- Subsegment: more granular — DB query, HTTP call within a segment
- Annotations: indexed key-value pairs (searchable in X-Ray console)
- Metadata: non-indexed key-value pairs (not searchable)

- Sampling: records a % of requests (default 5% + 1 req/sec reservoir)
 - Groups: filter traces with filter expressions
 - Use when: **"trace requests across microservices"** or **"find bottleneck in distributed app"**
-

4.4 Amazon OpenSearch Service

- Log analytics and full-text search
 - Common pattern: CloudWatch Logs → subscription filter → Kinesis Firehose → OpenSearch
 - Or: CloudWatch Logs → Lambda → OpenSearch
 - Use when: **"analyse logs"** + **"full-text search"** or **"visualise with Kibana/Dashboards"**
-

4.5 Kinesis Family

Kinesis Data Streams:

- Real-time data streaming, sub-second latency
- Shard-based scaling: 1 shard = 1MB/s in, 2MB/s out
- Retention: 24 hours default, up to 365 days
- Consumers: Lambda, KCL applications, Kinesis Data Analytics
- Use for: real-time processing, custom consumers

Kinesis Data Firehose:

- Managed delivery of streaming data to destinations
- Destinations: S3, Redshift, OpenSearch, Splunk, HTTP endpoint
- Near-real-time: 60 second buffer minimum
- Can transform with Lambda before delivery
- No consumer code needed — fully managed
- Use for: log delivery to S3/OpenSearch without managing consumers

Kinesis Data Analytics:

- SQL or Apache Flink on streaming data
- Use for: real-time aggregations, anomaly detection on streams

DOMAIN 5 — Incident & Event Response (14%)

5.1 EventBridge

The central nervous system for event-driven automation in AWS.

- Event bus: default (AWS services), custom (your apps), partner (SaaS)
- Rules: match events by pattern → route to targets
- Targets: Lambda, SNS, SQS, CodePipeline, Step Functions, ECS task, SSM Automation, + 20 more
- Scheduled rules: cron or rate expressions (replaces CloudWatch scheduled events)
- Schema registry: discovers and stores event schemas
- Pipes: point-to-point integration with filtering and enrichment

Key patterns:

Event	EventBridge Rule	Target
EC2 instance state change (stopped)	source: aws.ec2, detail-type: EC2 Instance State-change Notification	SNS → email
CodePipeline stage failure	source: aws.codepipeline, detail-type: CodePipeline Stage Execution State Change	Lambda → Slack
CodeCommit push to main	source: aws.codecommit, detail-type: CodeCommit Repository State Change	CodePipeline start
Config rule non-compliance	source: aws.config	SSM Automation
GuardDuty finding	source: aws.guardduty	Lambda → remediation
Scheduled (every day 2am)	rate(1 day) or cron(0 2 * * ? *)	Lambda → backup

5.2 Auto-Remediation Patterns

Pattern 1 — Config + SSM Automation: AWS Config Rule detects violation → triggers SSM Automation runbook → fixes resource Example: S3 bucket ACL set to public → Config detects → SSM doc makes private

Pattern 2 — CloudTrail + EventBridge + Lambda: API call detected by CloudTrail → EventBridge rule matches API name → Lambda remediates Example: Security group allows 0.0.0.0/0 on port 22 → EventBridge detects AuthorizeSecurityGroupIngress → Lambda revokes the rule

Pattern 3 — GuardDuty + EventBridge + Lambda: GuardDuty finding (e.g., compromised EC2) → EventBridge → Lambda → isolate instance (remove from ASG, quarantine SG)

Pattern 4 — Health Dashboard + EventBridge: AWS service health event → EventBridge → Lambda → notify on-call team

5.3 AWS Systems Manager — Incident Response

OpsCenter:

- Centralised view of operational issues (OpsItems)
- Auto-creates OpsItems from: CloudWatch alarms, Config rules, GuardDuty findings
- Links related resources and runbooks to each OpsItem

Incident Manager:

- Incident response with escalation plans, runbooks, and post-incident analysis
 - Integrates with PagerDuty, OpsGenie
-

5.4 Step Functions

- Serverless workflow orchestration
- States: Task, Choice, Wait, Parallel, Map, Pass, Succeed, Fail
- Standard workflows: exactly-once, up to 1 year, audit history
- Express workflows: at-least-once, up to 5 minutes, high throughput, cheaper
- Error handling: Retry and Catch at state level

- **Use when:** "orchestrate multi-step Lambda workflow" or "complex business process" or "handle failures across Lambda chain"
 - **Trap:** SQS + Lambda for simple async decoupling. Step Functions for complex multi-step workflows with branching, retries, parallel execution.
-

DOMAIN 6 — Security & Compliance (17%)

6.1 IAM Deep Dive

Policy evaluation logic:

1. Explicit Deny in any policy → DENY (always wins)
2. SCP allows the action → check further
3. Resource-based policy allows → ALLOW (for same account)
4. Identity-based policy allows → ALLOW
5. No allow found → implicit DENY

Policy types (in order of precedence):

- SCP (org level) — maximum permissions for account
- Permission boundary — maximum permissions for IAM entity (user/role)
- Resource-based policy — attached to resource (S3 bucket, SQS queue, Lambda)
- Identity-based policy — attached to user/group/role
- Session policy — passed when assuming role

Trust policy vs permission policy:

- Trust policy: who can ASSUME this role (principal)
- Permission policy: what this role can DO

Cross-account access pattern: Account A role assumes Account B role → Account B role has trust policy allowing Account A role ARN → Account B role has permission policy for needed actions

IAM Access Analyzer:

- Finds resources shared with external entities (outside your org/account)
 - Resources analysed: S3 buckets, IAM roles, KMS keys, Lambda, SQS, Secrets Manager
 - Use for: "find all S3 buckets accessible from outside my account"
-

6.2 AWS Organizations & SCP

SCP rules:

- SCP applies to all principals in the account EXCEPT the management account
- SCP does NOT apply to service-linked roles
- Root user in member account is also restricted by SCP
- Deny list strategy (recommended): allow * then add specific deny SCPs
- Allow list strategy: remove default FullAWSAccess then add specific allow SCPs

Common SCP use cases in exam:

- "Prevent disabling CloudTrail in any account" → SCP Deny cloudtrail:StopLogging
 - "Prevent leaving the organisation" → SCP Deny organizations:LeaveOrganization
 - "Restrict to specific regions only" → SCP Deny * with condition aws:RequestedRegion NotIn [allowed regions]
 - "Prevent creating IAM users (force SSO)" → SCP Deny iam:CreateUser
-

6.3 Secrets Manager vs Parameter Store

Feature	Secrets Manager	Parameter Store
Automatic rotation	Yes (native + custom Lambda)	No
Cost	\$0.40/secret/month + \$0.05/10K API calls	Standard: free. Advanced: \$0.05/param/month
Max size	65KB	4KB standard, 8KB advanced
Cross-region replication	Yes	No

Feature	Secrets Manager	Parameter Store
Database integration	RDS, Redshift, DocumentDB native	No
Versioning	Yes (staging labels: AWSCURRENT, AWSPENDING, AWSPREVIOUS)	Yes
KMS encryption	Always (required)	Optional (SecureString uses KMS)

Decision rule:

- "Automatic rotation" → Secrets Manager (always)
- "Cost-effective, no rotation" → Parameter Store SecureString
- "Same secret in two regions" → Secrets Manager replication
- "Hierarchy of config values" → Parameter Store paths (/app/env/key)

6.4 KMS

- Customer Master Keys (CMKs): AWS managed (free) or Customer managed (\$1/month/key)
- Key policies: resource-based policy on the key — must explicitly allow account
- Envelope encryption: KMS encrypts a data key, data key encrypts data (for large data)
- Key rotation: AWS managed keys rotate every year automatically. Customer managed: enable annual rotation.
- Multi-region keys: same key material in multiple regions (different ARNs, same key ID)
- **Use when: "encrypt data with customer-managed keys" or "audit key usage" or "cross-region encryption with same key"**

6.5 GuardDuty

- Intelligent threat detection using ML + threat intelligence
- Sources: VPC Flow Logs, DNS logs, CloudTrail events, S3 data events, EKS audit logs
- Findings: organized by severity (low/medium/high)

- **Does NOT prevent threats — only detects and alerts**
 - Remediation: GuardDuty finding → EventBridge → Lambda → isolate/remediate
 - Multi-account: delegate admin to one account, enable org-wide
 - **Use when: "detect compromised instances" or "identify unusual API activity" or "malware detection on EC2"**
-

6.6 Security Hub

- Centralised security findings across AWS accounts
 - Aggregates: GuardDuty, Inspector, Macie, IAM Access Analyzer, Config, Firewall Manager
 - Security standards: CIS AWS Foundations, AWS Foundational Security Best Practices, PCI DSS
 - **Use when: "single pane of glass for security" or "compliance score across org"**
-

6.7 Amazon Inspector

- Automated vulnerability assessment
 - For EC2: uses SSM Agent, checks OS CVEs and network exposure
 - For ECR: scans container images on push (enhanced scanning = continuous)
 - For Lambda: scans function code and dependencies
 - **Use when: "find vulnerabilities in EC2 instances/containers" or "CVE scanning"**
-

6.8 Amazon Macie

- Uses ML to discover and protect sensitive data in S3
 - Detects: PII, credentials, financial data
 - **Use when: "find sensitive data in S3" or "PII detection" or "data privacy compliance"**
-

PART 7 — The Big Exam Traps (Read Night Before Exam)

Trap 1: CloudWatch vs CloudTrail

- CloudWatch = metrics, logs, alarms. Operational monitoring. What IS happening.
 - CloudTrail = API audit log. Who DID what, when, from where.
 - "Monitor CPU" → CloudWatch
 - "Who deleted my S3 bucket" → CloudTrail
 - "Alert when CPU > 80%" → CloudWatch Alarm
 - "Alert when someone calls DeleteSecurityGroup" → CloudTrail + EventBridge
-

Trap 2: CodeDeploy in-place vs blue/green

- "Zero downtime" = blue/green. Always. No exceptions.
 - "Instant rollback" = blue/green.
 - "Save cost, some downtime ok" = in-place rolling.
-

Trap 3: Secrets Manager vs Parameter Store

- Automatic rotation = Secrets Manager. Always.
 - "Cost-effective storage, no rotation" = Parameter Store.
 - "Same secret in two regions" = Secrets Manager replication.
-

Trap 4: CloudFront vs Global Accelerator

- CloudFront = HTTP/S, caching, static content, dynamic content with cache-bypass.
 - Global Accelerator = any protocol, no caching, static IPs, gaming/IoT.
 - "Static website low latency globally" → CloudFront.
 - "Static IPs required for whitelisting" → Global Accelerator.
 - "Non-HTTP application HA" → Global Accelerator.
-

Trap 5: SCP vs IAM Policy

- SCP = what an account is ALLOWED to do at maximum. Does NOT grant.

- IAM = grants permissions within SCP boundary.
 - "Prevent any account from disabling CloudTrail" → SCP.
 - "Allow developers to launch EC2" → IAM policy.
 - SCP does NOT affect management account. Ever.
-

Trap 6: EventBridge vs SNS vs SQS

- EventBridge = react to AWS service events with rules and routing.
 - SNS = fan-out, notify multiple subscribers at once.
 - SQS = queue, guaranteed processing, decouple producer/consumer.
 - "React to EC2 state change" → EventBridge.
 - "Notify 5 teams simultaneously" → SNS.
 - "Process jobs reliably, no message loss" → SQS.
 - "Fan-out to multiple SQS queues" → SNS → multiple SQS subscriptions.
-

Trap 7: Aurora Global vs DynamoDB Global Tables

- Aurora Global = relational, one write region, reads from secondary regions, failover < 1 min.
 - DynamoDB Global Tables = NoSQL, multi-master, write from ANY region.
 - "Relational DB + multi-region + read scale" → Aurora Global.
 - "Write from multiple regions simultaneously" → DynamoDB Global Tables.
-

Trap 8: ElastiCache and DAX are regional

- ElastiCache cannot cache across regions natively.
 - DynamoDB DAX is regional even though DynamoDB itself can be global.
 - For global caching: CloudFront (HTTP) or application-level logic.
-

Trap 9: ACM Certificates for CloudFront must be in us-east-1

- CloudFront is a global service but reads SSL certificates from us-east-1 only.
- For ALB/API Gateway in other regions: request cert in that region.

- "HTTPS on CloudFront" → request ACM cert in us-east-1 regardless of where your origin is.
-

Trap 10: Beanstalk blue/green vs CodeDeploy blue/green

- Beanstalk blue/green = swap environment URLs (CNAME swap at DNS level).
 - CodeDeploy blue/green = swap ALB target groups (ALB level, faster).
 - Both achieve zero downtime but the mechanism is different.
 - Exam will try to mix them up in scenarios.
-

Trap 11: CodePipeline is regional

- The pipeline runs in one region.
 - To deploy to multiple regions: use cross-region deployment actions in CodePipeline.
 - CodePipeline creates replication S3 buckets in each target region automatically.
-

Trap 12: Parameter Store does not cross regions

- If question says "same config value available in two regions" → cannot be Parameter Store.
 - Answer: Secrets Manager with cross-region replication OR store separately in each region.
-

PART 8 — Last-Minute Scenario Quick-Fire

Read these the morning of your exam. Each one is a real pattern.

Q: GitHub push triggers pipeline, deploys to ECS, requires approval before prod. A: GitHub → CodeStar Connections → CodePipeline → CodeBuild → Manual Approval (SNS) → CodeDeploy ECS blue/green

Q: EC2 without required tags auto-remediated. A: AWS Config required-tags rule → SSM Automation document adds tags

Q: Deploy same CloudFormation stack to 50 accounts, 3 regions. A: CloudFormation StackSets with Organizations SERVICE_MANAGED mode

Q: Someone deleted a security group — detect and notify. A: CloudTrail → EventBridge rule (DeleteSecurityGroup API) → SNS → email

Q: Asia users have high latency, app in us-east-1, static content. A: CloudFront with edge caching

Q: Asia users have high latency, app in us-east-1, dynamic content. A: Route 53 latency routing + deploy stack in ap-southeast-1

Q: Need static IPs for firewall whitelisting, global HA. A: Global Accelerator (two static anycast IPs)

Q: Lambda needs different config per environment without code changes. A: SSM Parameter Store with path hierarchy (/app/dev/key, /app/prod/key)

Q: Lambda config deployment with validation and rollback capability. A: AppConfig with JSON Schema validator and deployment strategy

Q: Rotate RDS password automatically every 30 days. A: Secrets Manager with RDS rotation Lambda (native integration)

Q: Centralised audit logs from all accounts and regions. A: CloudTrail Organization Trail → central S3 bucket in security account

Q: EC2 patching without SSH, with compliance reporting. A: SSM Patch Manager with maintenance windows + patch compliance dashboard

Q: Run a script on 500 EC2 instances simultaneously. A: SSM Run Command with rate control (concurrency %)

Q: Detect compromised EC2 instance, auto-isolate. A: GuardDuty finding → EventBridge → Lambda → remove from ASG + apply quarantine SG

Q: Find PII data stored in S3 buckets. A: Amazon Macie

Q: Vulnerability scan for all EC2 instances continuously. A: Amazon Inspector (with SSM Agent on instances)

Q: Single view of all security findings across 100 AWS accounts. A: Security Hub with organisation-wide delegation

Q: Multi-region relational DB, < 1 min failover, sub-second replication. A: Aurora Global Database

Q: Multi-region NoSQL, write from any region, low latency. A: DynamoDB Global Tables

Q: Prevent any account in org from creating IAM users (force SSO only). A: SCP with Deny iam:CreateUser on org root

Q: Canary deployment — 10% traffic to new Lambda version, rollback if errors. A: CodeDeploy Lambda deployment group with Canary10Percent5Minutes + CloudWatch alarm

Q: Application config change must be validated and rolled back if Lambda test fails. A: AppConfig with Lambda validator + rollback on CloudWatch alarm

Q: Stream application logs to OpenSearch for real-time analysis. A: CloudWatch Logs → subscription filter → Kinesis Firehose → OpenSearch

Q: Detect unusual API call patterns (possible account compromise). A: CloudTrail Insights → EventBridge → SNS

Q: EC2 instances must always have CloudWatch agent installed (enforce continuously). A: SSM State Manager association with CloudWatch agent document on a schedule

End of DOP-C02 Study Guide Good luck on your exam. You've got this.